

**INDIAN INSTITUTE OF
INFORMATION TECHNOLOGY
GUWAHATI**

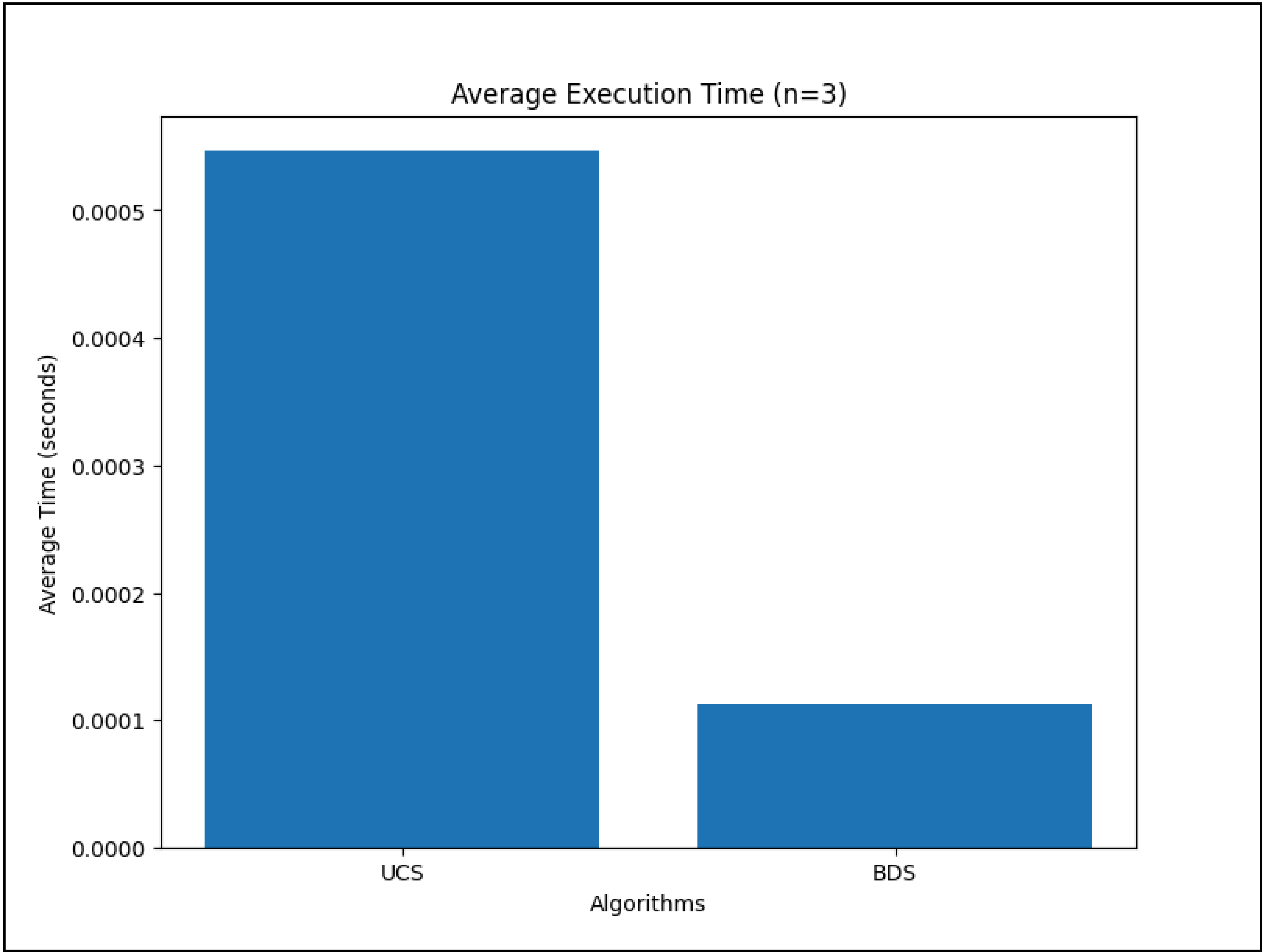


**AI LAB
Assignment-5**

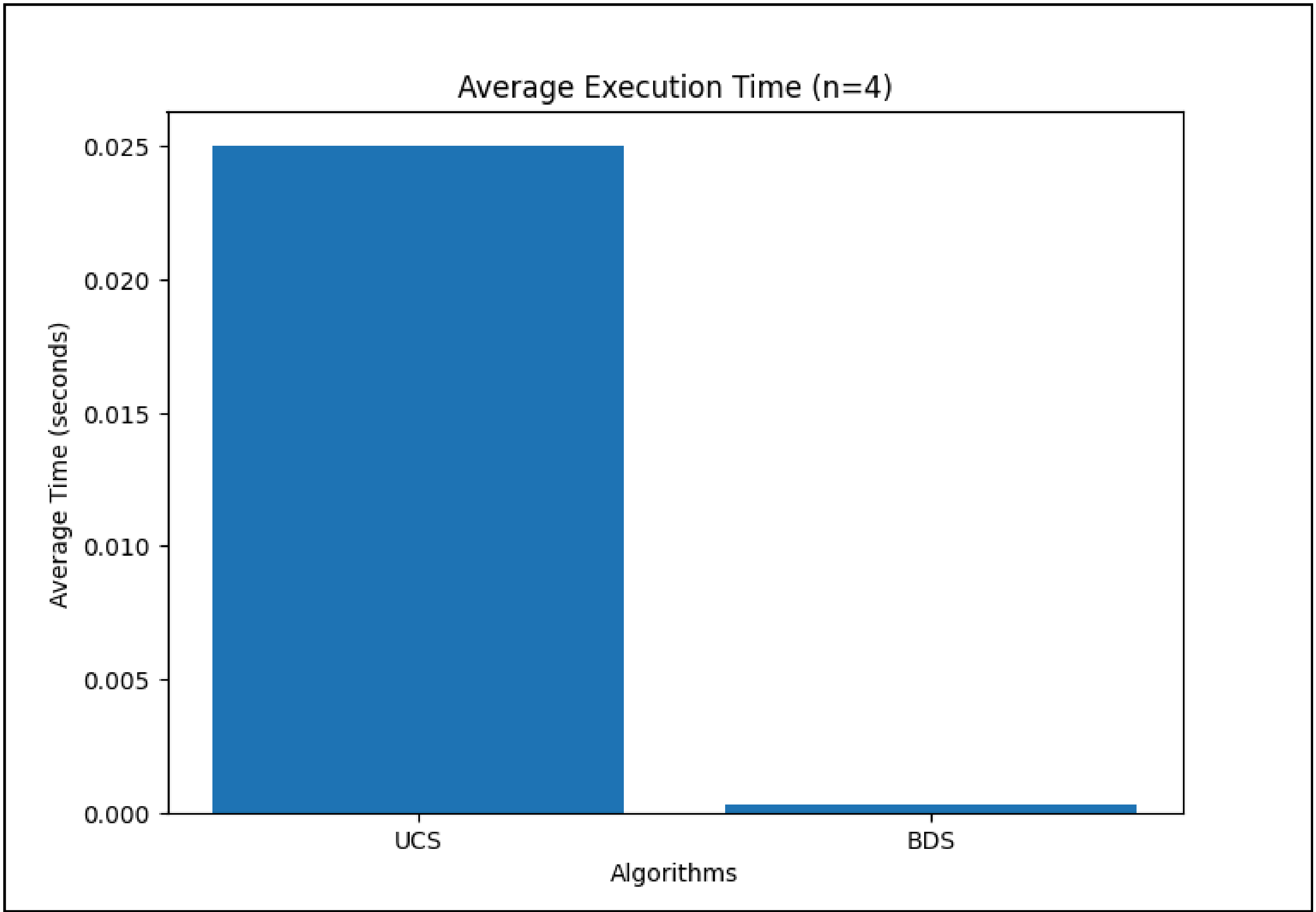
Name: Ansik Singh Tomar
Roll No: 2401037
Section: CS21
Branch: B.Tech 2nd Year CSE
Course: Artificial Intelligence Lab
Code: CS236

Average Execution Time

For n = 3

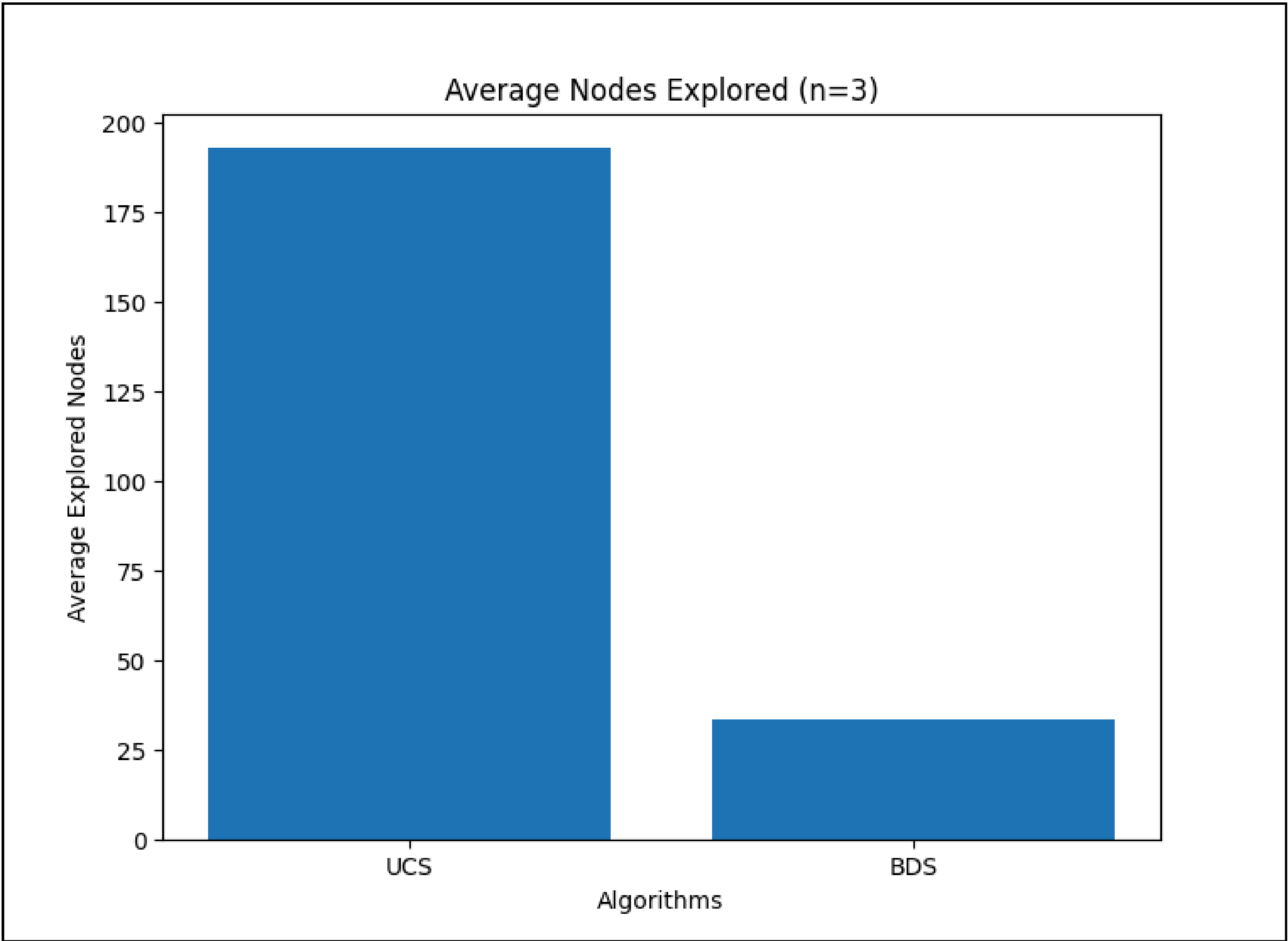


For n = 4

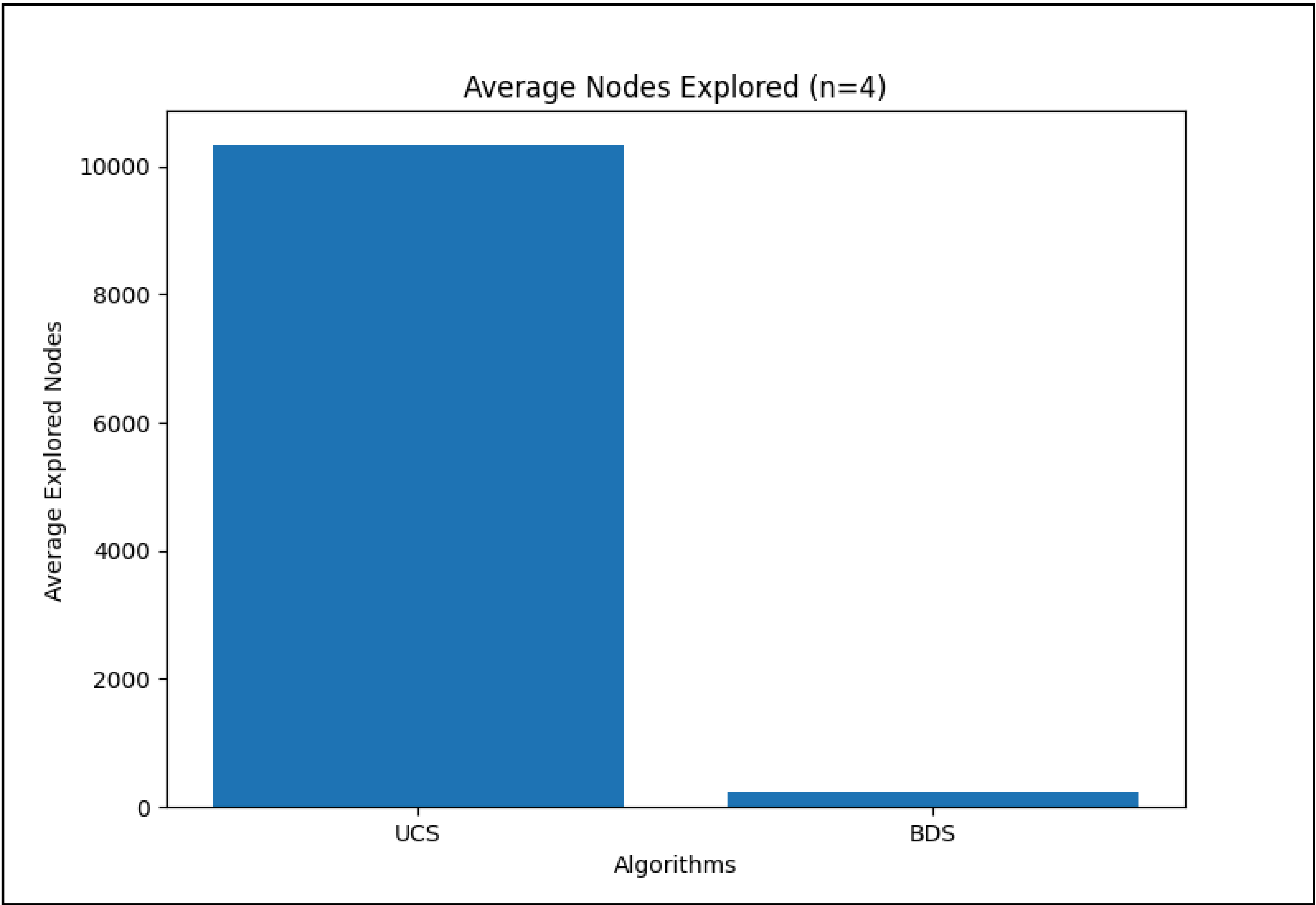


Average Explored Nodes

For n = 3



For n = 4



1. A brief explanation of how you implemented the meeting condition for the Bidirectional Search

Firstly, we will maintain two priority queue and two visited array with the node and their depth, for both the forward and backward directions. whenever we explore a node we create all its possible child nodes (moves) and check, if that node (move) is not already explored i.e. not in the forward/backward visited array, then we will check if it is present in the opposite direction's visited array, if it is present then we found the intersection and add the current depth and depth of node stored in opposite array to return the overall depth from initial to goal.

2. An explanation of what happened to the success rate when moving from $n = 3$ to $n = 4$.

The primary reason is due to the search space complexity. If the solution is at depth d and the branching factor is b , UCS explores roughly $O(b^d)$ nodes. while in case of bidirectional Search, by meeting in the middle, each search only needs to travel a distance of $d/2$. This results in a complexity of $O(b^{\{d/2\}})$. Hence search space is reduced drastically in Bidirectional Search. thus it is faster